

M16 / STUDY COMPANION

M16 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

DE16-01 | Production mindset 3

DE16-02 | Testing strategy for data engineering 3

DE16-03 | Automated quality gates 3

DE16-04 | Continuous Integration fundamentals 3

DE16-05 | GitHub Actions for data projects 3

DE16-06 | Dev, test and prod environments 4

DE16-07 | Deployment and rollback 4

DE16-08 | Secrets and credential safety 4

DE16-09 | RBAC and least privilege 4

DE16-10 | PII, encryption and masking 4

DE16-11 | Metadata, lineage and governance 4

DE16-12 | Observability for data pipelines 5

DE16-13 | Incidents, runbooks and root cause 5

DE16-14 | Terraform and IaC foundations 5

Read this first

Ready-made retrieval notes. Use after normal study; do not rewrite the learner book.

DE16-01 | Production mindset

Say it: A pipeline that works once is a prototype.

Failure anchor: At 05:40 the orchestration UI says success, but the gold table contains only 12% of the normal rows because the source API changed pagination.

Teach-back: Explain why 'the DAG is green' is not enough to call a data product healthy. Give two examples of successful execution with wrong business output.

DE16-02 | Testing strategy for data engineering

Say it: Data systems fail at many layers: a pure function can be wrong, a schema can drift, a database boundary can misbehave, a join can change grain, or totals can stop reconciling.

Failure anchor: A refactor passes 65 unit tests but doubles daily revenue because a join changed grain and created fan-out.

Teach-back: Using one orders example, distinguish a unit test, a data-quality expectation, an integration test and a reconciliation check.

DE16-03 | Automated quality gates

Say it: A validation report becomes operationally useful only when its result controls a decision.

Failure anchor: A source sends half the normal rows after an upstream timeout.

Teach-back: Why is a validation report alone not a quality gate? Name the additional control that makes it a gate.

DE16-04 | Continuous Integration fundamentals

Say it: Continuous Integration (CI) means small changes are integrated frequently and checked automatically before they damage the shared codebase.

Failure anchor: Common mistakes are running CI only after merge, hiding errors with continue-on-error, using floating dependencies, or putting powerful production credentials into a test workflow that never needed them.

Teach-back: Explain CI to someone who believes it means 'automatically deploy every commit.' What is missing from that definition?

DE16-05 | GitHub Actions for data projects

Say it: GitHub Actions is one practical CI/CD engine for repository events.

Failure anchor: A workflow uses a long-lived cloud key stored as a secret even though the provider supports OIDC.

Teach-back: What is the difference between `uses:` and `run:`? Why should workflow permissions be designed explicitly?

DE16-06 | Dev, test and prod environments

Say it: Environment separation limits blast radius and makes promotion reviewable.

Failure anchor: A test pipeline is copied manually from production and still references the prod sink.

Teach-back: Why should promotion prefer the same artifact with different bindings rather than three manually edited copies?

DE16-07 | Deployment and rollback

Say it: Deployment changes running behavior; rollback is the controlled path back when a release proves unsafe.

Failure anchor: A destructive schema migration drops a column before all consumers are updated.

Teach-back: Why is code rollback not the same as data rollback? Give one example where the old code is restored but the system remains wrong.

DE16-08 | Secrets and credential safety

Say it: Credentials are authority.

Failure anchor: A key is accidentally committed and then 'fixed' by deleting the line in the next commit.

Teach-back: Why does deleting a leaked credential from the latest source file not make the credential safe again?

DE16-09 | RBAC and least privilege

Say it: Role-based access control (RBAC) limits who can do what at which scope.

Failure anchor: A pipeline receives 403 and the quickest fix is to grant Owner at subscription scope.

Teach-back: Explain principal + role + scope with one Azure example. Why can a management-plane role differ from data access?

DE16-10 | PII, encryption and masking

Say it: Personal data creates a different risk class.

Failure anchor: A pipeline correctly encrypts storage but prints full email addresses and authentication headers to logs during debugging.

Teach-back: Why are encryption and masking different controls? Give one failure that encryption alone does not prevent.

DE16-11 | Metadata, lineage and governance

Say it: When a field is wrong, teams need to know where it came from, what transformed it, who owns it, what consumes it and whether it is sensitive.

Failure anchor: A catalog lists every table but no owners, transformation relationships or sensitive classifications.

Teach-back: What can lineage answer that a schema alone cannot? Give one upstream root-cause and one downstream impact question.

DE16-12 | Observability for data pipelines

Say it: Monitoring tells you that a metric crossed a threshold; observability helps you understand system state from evidence.

Failure anchor: Teams emit `customer_email` as a metric label to debug individual failures.

Teach-back: Why can CPU/runtime/success metrics all be healthy while a data product is unhealthy? Name three data-specific signals.

DE16-13 | Incidents, runbooks and root cause

Say it: Production incidents are time-constrained reasoning problems.

Failure anchor: During an incident someone reruns the pipeline repeatedly with debug flags and deletes the failed batch to 'start clean'.

Teach-back: What is the difference between a runbook and an RCA? Why should facts and hypotheses be labeled separately during an incident?

DE16-14 | Terraform and IaC foundations

Say it: Infrastructure as Code (IaC) expresses infrastructure intent in versioned files so changes can be reviewed, repeated and compared rather than reconstructed from memory and portal clicks.

Failure anchor: A team edits production resources manually after Terraform deploy.

Teach-back: Explain why `terraform plan` is valuable but does not prove application or data correctness. What extra evidence belongs after infrastructure deployment?